

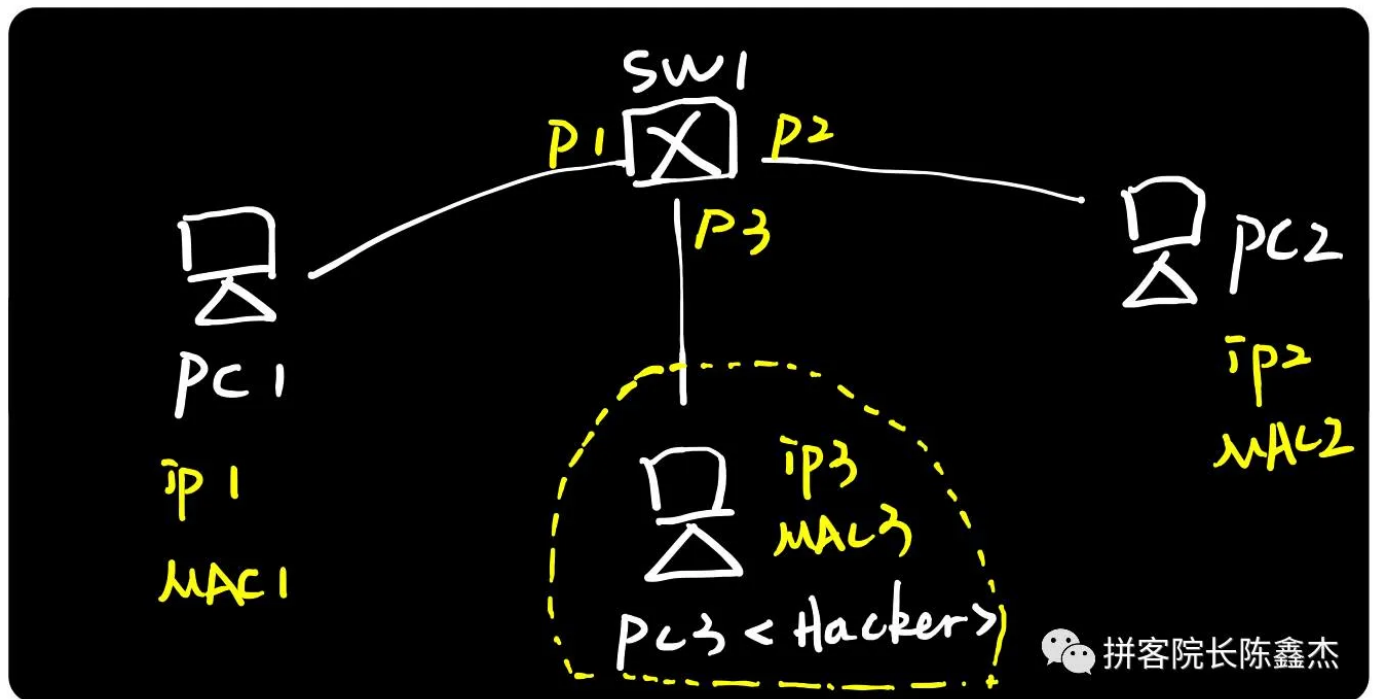
Hack My Robot

Attack 1: DoS

1.1 ARP Spoofing

Requirements: This Attack need us to connect into the wifi.

ARP (Address Resolution Protocol) is widely used in IPv4 networks. Simply put, every device in a computer has a MAC address, which marks the physical address of the device. Also each host has an ARP table to record the mapping relationship between IP and MAC addresses. arp, is to create a mapping from IP address to MAC address, IP layer is used for network, while MAC layer is used for LAN communication. But unfortunately APR protocol is not a secure protocol. It relies on each host to honestly broadcast to the LAN, telling everyone their IP address and MAC address mapping. So if there is some malicious sender that claims some IP address that does not belong to it belongs to itself, so this malicious broadcast message can pollute the whole LAN. If this malicious sender sends messages densely enough, then these malicious messages will overwrite the MAC address of the victim and make other hosts in the network believe that this malicious host is the victim. This is the principle of ARP attack.

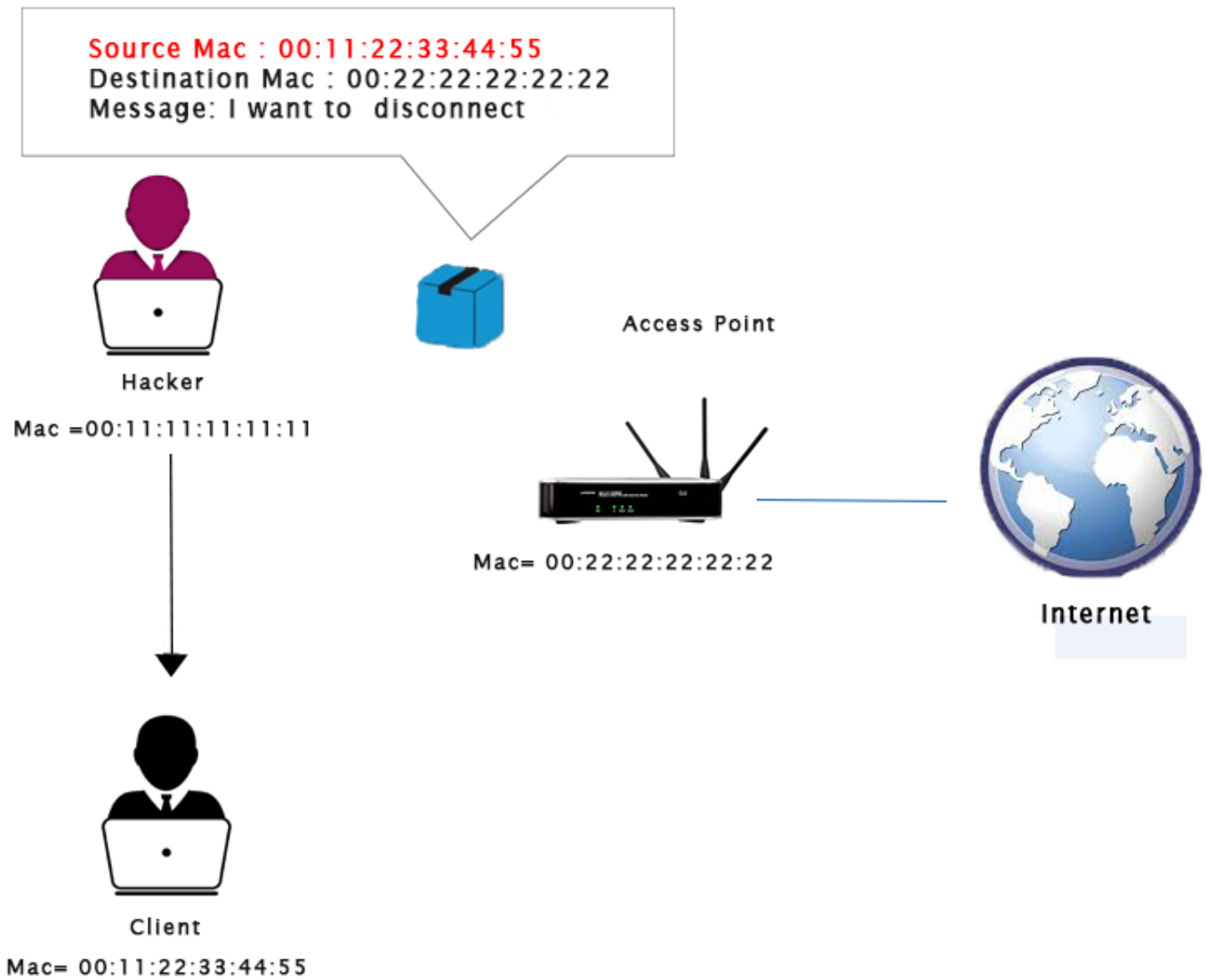


What ARP Spoofing does is to pollute the MAC table of other hosts by broadcasting malicious ARP packets to achieve the purpose of disguise. ARP Spoofing is simple, it only needs to send malicious packets to both the host and the robot, so that all the traffic flows through the attacker's host, in this way, the attacker will be able to play the role of controlling the host and robot's traffic. If you want to implement ARP DOS, you only need to intercept the traffic of both sides to block the communication between the host and robot.

1.2 Deauth Attack

Requirements: Nothing

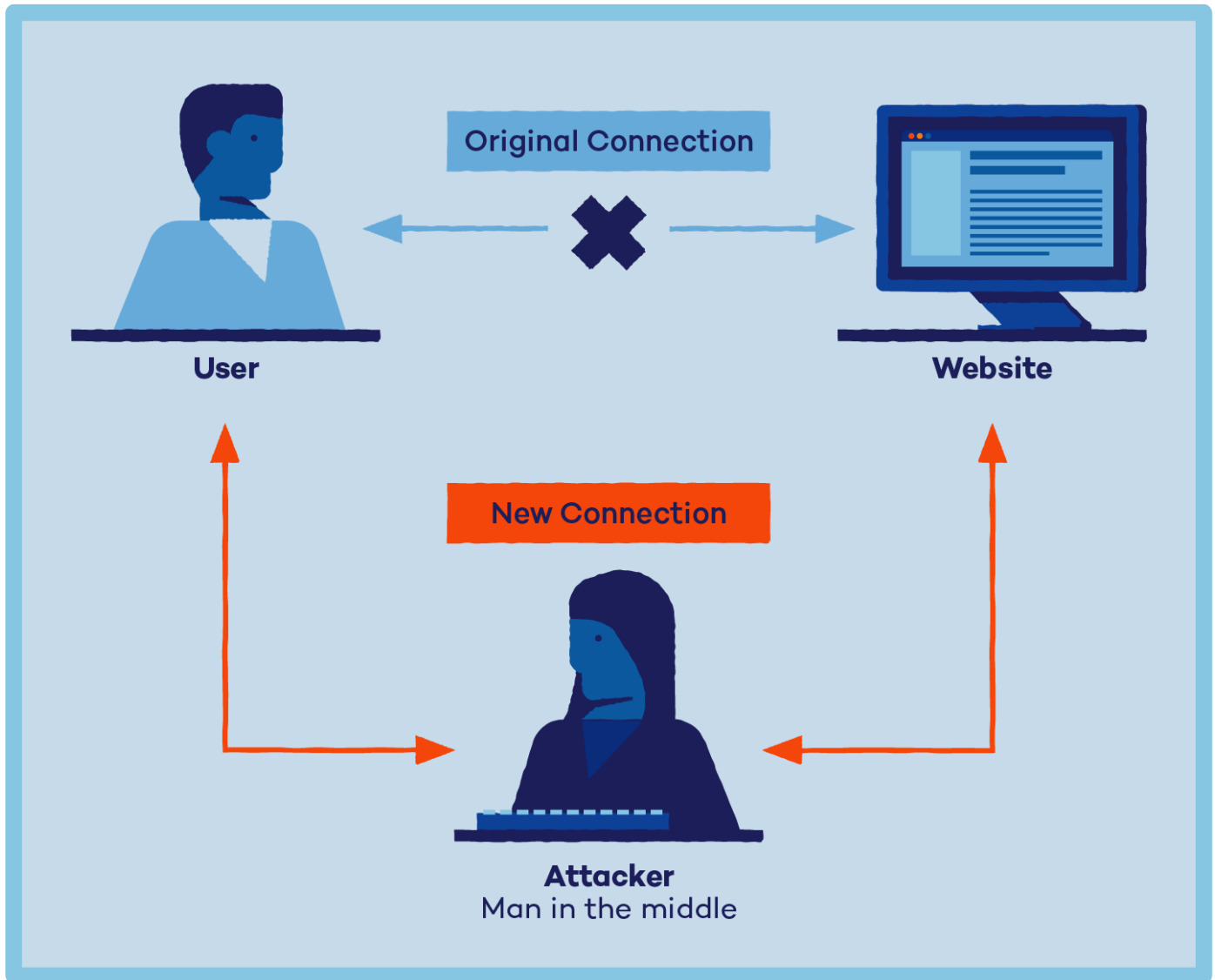
In WiFi, there are special packets that can control the behavior of the network hosts. Therefore we can use a listening NIC to get the MAC address of the router and that host, and subsequently broadcast and send a large number of **Deauth** attack packets to force the wifi communication between the bot and the host to disconnect for DOS purposes. Subsequently we can capture the handshake packets of the robot and the router to re-communicate for WiFi password acquisition



Attack 2: Packet Inspection

Requirements: Successful ARP Spoofing

Once we have the APR attack as a base, we can modify on the network traffic and complete a TCP hijacking based on ARP attack. We observed through **wireshark** packet capture that the data communication between robot and the host is transmitted via TCP, and it is unencrypted TCP, which gives us an opportunity to take advantage of it. Through **wireshark**, we found that the host sends a packet with linear and angular velocities to the robot at regular intervals to specify the speed of the cart. So if we can modify these packets, we can control the speed of the cart. By reading the source code of telelop, we found that the packet format has a certain pattern, the 4th-12th bytes of the packet represent the line speed and the last 8 bytes represent the angular speed. So by modifying these data, we can control the movement of the car.



We use python's `scapy` and `nfqueue` to accomplish this attack, using python to spoof packets and `nfqueue` and `iptables` to transfer packets from kernel state to user state for the purpose of blocking and modifying packets.

Attack 3: Backdoor

3.1 Get Root access

Requirement: Physical Access

Since all the data of the Raspberry Pi is stored on the SD card, we can pull out the SD card, read the `/boot/cmdline.txt` file of the linux kernel boot, add the `init=/bin/sh` command in the last line, and then put the SD card back into the Raspberry Pi, so that the Raspberry Pi will start a shell for user operation in priority when booting . In the new shell, we can then run the `passwd` command to change the passwords of all users and gain full control of the system

3.2 meterpreter/backdoor

When we had a shell, we needed a bouncing shell that would allow us to control the robot. we used the `metasploit` suite `meterpreter` to generate a python-based backdoor program. When the backdoor runs, the program automatically opens a shell and connects to the attacker's machine to generate a `session` for the attacker to execute commands on the bot. We then write the program to the bot's `crontab` and have it automatically connect to the host machine. In this way, we can persist a backdoor program that allows us to achieve full control over the bot

3.3 FRP Port Forwarding

It is possible that the bot is on a LAN and sometimes we don't know the IP address of the bot or the IP address of the bot is in a huge intranet and we can only access the bot if we know the IP address of the bot. So we use `frp` to run port forwarding. When the bot starts, the bot will actively connect to the attacker's machine, communicate with the attacker's machine, and map its port 22 to port 6000 of the attacker's machine, so that the attacker only needs to connect to port 6000 of his machine instead of the bot's IP address to access the bot using ssh. frp does not require the bot to have a public IP.

3.4 Key Information Extraction

Because of the root access and the backdoor, we can do whatever we want to do. For example, to get the wifi password, we just need to read `/etc/wpa_supplicant/wpa_supplicant.conf`, and we can know the password of the wifi access point. By writing the public key file to `.ssh/authorized_keys`, we can login without password. By reading the source code and using the `rsync` command, we can send arbitrary files from the bot to the attacker's machine